# 314581030 徐煜宸 HW4 Report

## AI prompt

- 這份作業我先自己閱讀 HW4 PDF，確認四個題目分別是 stable matching、stability checker、fairest stable matching，以及 many-to-one matching。
- 我希望這次仍然以 Python 為主，並且延續 HW1-HW3 的方式：先和 AI 討論整體架構與演算法方向，再讓 Codex 依照資料格式實作完整程式，最後我再看 code 和 report 是否符合我的理解。
- 其中 Problem 3 因為有 bonus opportunity，所以我有特別和 AI 討論不要只做 naive brute force，而是用 branch-and-bound 加 stability pruning 的方式，讓 report 可以清楚說明 optimization strategy。
- 後來因為 12! 雖然很大但仍然可以完整枚舉，所以我又請 AI 額外寫一個獨立的 hw3_checker.py，用 permutation brute force 驗證 Problem 3 的 pruning solver 沒有剪掉真正最佳解。
- Report 的部分也是請 AI 先根據 HW4 PDF、HW4_DATA/、目前 src/main.py 的內容，以及跑出來的四個 CSV，整理成符合我前幾次作業風格的版本，後面再由我人工修正。

大致的 prompt 是：

> 請你到 314581030_HW4 下，查看 HW4 PDF、HW4_DATA、以及我 HW1-HW3 的 report 和 src，
> 先理解整個作業目標。確認後用 Python 實作四題 matching problem，
> 再幫我整理一份 report，內容要包含 code explanation、optimization strategy、
> result analysis、verification，以及 AI prompt description。

---

## 作業目標與整體說明

這次 HW4 的主題是 matching theory，核心是把課堂上 stable matching 和 Deferred Acceptance algorithm 的概念轉成實際可執行的程式。

整份作業分成四個 problem：

1. Problem 1: Basic Stable Matching

- 實作 men-proposing Deferred Acceptance algorithm。
- 輸入為 HW1_men_prefs.csv 和 HW1_women_prefs.csv。
- 輸出一份 man_id,woman_id 格式的 matching。

2. Problem 2: Stability Checker

- 對助教提供的三組 matching 檢查是否 stable。
- 如果 unstable，就列出所有 blocking pairs。
- blocking pair (m, w) 的意思是：man m 和 woman w 都比目前 partner 更喜歡彼此。

3. Problem 3: Finding the Fairest Stable Matching

- 在 stable matchings 裡面找 total rank sum 最小的 matching。
- rank score 是從 0 開始，也就是第一志願分數為 0，第二志願分數為 1。
- 這題如果使用 non-brute-force optimization strategy，report 中說明清楚可以拿 bonus。

4. Problem 4: Many-to-One

- 實作 student-proposing Deferred Acceptance。
- 每個 school 有 quota = 10。
- 學校可以同時 hold 多位學生，但只能保留自己目前最喜歡的 10 位。

本次實作的入口是：

```
python3 -B 314581030_HW4/src/main.py
```

程式會讀取：

```
314581030_HW4/HW4_DATA/
```

並產生：

```
314581030_HW4/src/314581030_HW4_p1.csv
314581030_HW4/src/314581030_HW4_p2.csv
314581030_HW4/src/314581030_HW4_p3.csv
314581030_HW4/src/314581030_HW4_p4.csv
```

---

# Code Explain

## main.py 整體架構

這次我把四題都集中在 src/main.py，主要原因是提交時比較不容易漏檔，而且這份作業的共用邏輯其實都圍繞在 preference list、rank map、matching validation 這幾個工具上。

另外我有放一個 src/hw3_checker.py，這支不是正式 solver，而是用來驗證 Problem 3 的獨立 checker。它不 import main.py 裡的 find_fairest_stable_matching()，而是直接讀 preference list 和 314581030_HW4_p3.csv，用 permutation brute force 檢查答案是否真的是所有 stable matchings 中 rank sum 最小的。

整體流程大概是：

```
def solve_all(data_dir, output_dir, quota):
    # Problem 1
    # read HW1 preference lists
    # run men-proposing DA
    # write p1 csv

    # Problem 2
```

```
# read HW2 preference lists and matching cases
# find all blocking pairs
# write p2 csv-like text output

# Problem 3
# read HW3 preference lists
# search fairest stable matching
# write p3 csv

# Problem 4
# read student/school preference lists
# run many-to-one DA with quota = 10
# write p4 csv
```

在資料讀取上，我把 preference list 都讀成：

```
{
  agent_id: [candidate_1, candidate_2, candidate_3, ...]
}
```

也就是 CSV 每一列的第一個數字是 agent ID，後面照順序代表他的 preference ranking。

為了讓比較排名更快，我另外建立 rank_map：

```
{
  agent_id: {
    candidate_id: rank
  }
}
```

這樣如果要判斷 woman w 比較喜歡 m1 還是 m2，就可以直接比較：

```
women_rank[w][m1] < women_rank[w][m2]
```

rank 越小代表越喜歡。

---

# Problem 1: Men-Proposing Deferred Acceptance

## 演算法邏輯

Problem 1 使用標準 men-proposing DA algorithm。

我的實作流程是：

1. 一開始所有 men 都是 free。
2. 每個 free man 依照自己的 preference list，向目前還沒提過的最高順位 woman proposal。
3. 如果 woman 目前沒有人，她就先 hold 這位 man。
4. 如果 woman 目前已經 hold 一位 man，她會比較新 proposer 和目前 held man。
5. woman 只保留自己比較喜歡的那一位，另一位重新變成 free。

6. 重複直到沒有 free man。

這個過程結束後，每個 man 都會被 match 到一位 woman，而且結果會是 men-optimal stable matching。

### 輸出格式

輸出為：

```
man_id,woman_id
0,22
1,25
...
```

這份結果存在：

```
src/314581030_HW4_p1.csv
```

---

## Problem 2: Stability Checker

### Blocking Pair 定義

對一組 matching 來說，如果存在一對 (m, w) 滿足：

1. man m 比起自己目前 matched woman，更喜歡 woman w。
2. woman w 比起自己目前 matched man，更喜歡 man m。

那 (m, w) 就是一組 blocking pair。只要有任何 blocking pair，這組 matching 就是不 stable。

### 實作方式

我先把 matching 轉成兩個方向：

```
man_to_woman = {m: w}
woman_to_man = {w: m}
```

接著對每一個 man，只需要檢查他 preference list 中「排在目前 partner 前面」的 women。因為排在目前 partner 後面的 women，man 本人就不會想換過去，所以不可能形成 blocking pair。

概念上是：

```
for man in men:
    current_woman = matching[man]
    for woman in men_prefs[man]:
        if woman == current_woman:
            break
        current_man = woman_to_man[woman]
        if women_rank[woman][man] < women_rank[woman][current_man]:
```

```
        blocking_pairs.append((man, woman))
```

最後輸出格式照助教 example：

```
Matching_1: STABLE
Blocking Pairs: []

Matching_2: UNSTABLE
Blocking Pairs: [...]
```

我有注意到助教給的 HW2_solution_example..csv 比較像是輸出格式範例，而不是目前這組 data 的正確答案。因此這題沒有直接參考 example 的內容，而是重新根據 HW2_m_prefs_10x10.csv、HW2_w_prefs_10x10.csv 和 HW2_matching_cases.csv 計算 blocking pairs。

---

# Problem 3: Fairest Stable Matching

## Total Rank Sum

Problem 3 要找的是 stable matching 中 total rank sum 最小的那一個。

對每一對 matched pair (m, w)：

```
score(m, w) = rank_m(w) + rank_w(m)
```

整個 matching 的 total rank sum 是所有 pair 的 score 加總：

```
total_rank_sum = sum(score(m, w) for all matched pairs)
```

rank 是從 0 開始，所以如果某人 match 到第一志願，該側 score 就是 0。

## 為什麼不能只暴力枚舉

這題的資料是 12x12，如果直接枚舉所有 perfect matchings，數量是：

```
12! = 479001600
```

這在 Python 裡雖然不是完全不可能，但非常不漂亮，而且沒有利用 stable matching 的結構。更重要的是，PDF 特別提到 non-brute-force algorithm 可以有 bonus，所以我不希望這題只寫成 naive permutation search。

## Optimization Strategy

我使用的是 branch-and-bound + stability pruning。

整體想法是：

1. 先跑一次 men-proposing DA 和 women-proposing DA。

2. 這兩個結果一定是 stable matching，所以可以先拿 rank sum 比較小的那一個當作 initial upper bound。
3. 接著用 DFS 建構 matching，每次固定一位 man 的 woman。
4. 在 DFS 過程中，如果 partial matching 已經不可能變成 stable，就直接剪枝。
5. 如果目前 partial matching 的 optimistic lower bound 已經不可能比 best answer 更好，也直接剪枝。

## Stability Pruning

DFS 還沒排完所有人時，有些 blocking pair 已經可以被確定，或是確定未來也無法修復。

我主要做三種檢查：

1. 已固定 pair 之間已經互相 blocking

- 如果兩邊都已經被 match，而且他們互相更喜歡彼此，那後面不管怎麼配都救不了。

2. 已固定 man 和未固定 woman 可能 blocking，但 woman 未來沒有更好的 remaining man 可以保護

- 如果 man m 比目前 partner 更喜歡 woman w，而 w 未來也不可能拿到比 m 更喜歡的 remaining man，那這個 partial matching 不可能 stable。

3. 已固定 woman 和未固定 man 可能 blocking，但 man 未來沒有更好的 unused woman 可以保護

- 這是第二點的對稱情況。

這些檢查的目的不是直接求答案，而是在 DFS 搜尋樹還很小的時候排除不可能 stable 的分支。

## Bound Pruning

除了 stability pruning，我還用了 rank sum 的 lower bound。

對於目前已經固定的 pairs，score 已經確定。對於還沒 match 的 men，我先樂觀地假設他們都能拿到目前 unused women 中 score 最低的那一位，得到一個 optimistic lower bound。

如果：

```
lower_bound >= current_best_rank_sum
```

那代表這個分支即使在最樂觀狀況下，也不可能比目前答案更好，所以可以直接剪掉。

這邊使用 >= 的原因是作業只需要輸出 fairest matching 本身，不需要列出所有同分 matching。如果某個分支最多只能追平目前答案，就不用繼續展開。

## 本次搜尋結果

本次 Problem 3 的結果：

```
P3 stable: True
P3 fair rank sum: 59
visited = 498
stability_prunes = 352
bound_prunes = 85
```

這表示 DFS 實際訪問的節點只有 498 個，和 naive 12! 比起來差很多。這也是我認為可以在 report 中作為 non-brute-force optimization strategy 的主要說明。

輸出 matching 存在：

```
src/314581030_HW4_p3.csv
```

### Brute-force Checker 驗證

因為 Problem 3 的 size 是 12x12，所有 possible perfect matchings 數量是：

```
12! = 479001600
```

這個數量很大，但仍然可以用 Python 跑完整枚舉來當作離線驗證。因此我另外寫了 src/hw3_checker.py，直接枚舉所有 permutation，對每一個 matching 檢查是否 stable，若 stable 就計算 total rank sum。

這支 checker 的目的不是拿來當正式解法，而是確認 branch-and-bound solver 沒有因為 pruning 寫錯而漏掉更好的 stable matching。

完整 brute-force checker 的結果是：

```
checked_permutations = 479001600
stable_matchings = 5
brute_cost = 59
solution_cost = 59
brute_matching = {0: 3, 1: 8, 2: 6, 3: 7, 4: 9, 5: 1, 6: 11, 7: 2, 8: 5, 9: 0, 10: 10, 11:
 4}
```

所以目前 314581030_HW4_p3.csv 的答案和 brute-force optimum 完全一致。這讓我比較確定 Problem 3 的 pruning 沒有錯剪掉最佳解。

---

## Problem 4: Many-to-One Matching with Quotas

### 題目設定

Problem 4 是 many-to-one stable matching。

- student 數量：50
- school 數量：5
- 每個 school quota：10

所以總 capacity 剛好是：

```
5 * 10 = 50
```

每個 student 最後都會被分配到一間 school，每間 school 最多收 10 位 student。

### Student-Proposing DA

這題使用 student-proposing DA。

流程是：

1. 一開始所有 students 都是 free。
2. 每個 free student 向自己目前還沒提過、排名最高的 school proposal。
3. school 把新 proposer 加進目前 hold 的學生集合。
4. school 依照自己的 preference list 排序，只保留前 10 名。
5. 被擠出 quota 的學生重新變成 free，下一輪繼續向下一間 school proposal。
6. 重複直到沒有 free student。

實作上，每間 school 的暫存 list 會在加入新學生後排序：

```
held_students[school].sort(key=lambda candidate: school_rank[school][candidate])
```

如果超過 quota，就把排名最差的學生 pop 出來：

```
while len(held_students[school]) > quota:
    rejected = held_students[school].pop()
    free_students.append(rejected)
```

最後輸出格式為：

```
school_id,student_id
0,2
0,24
...
```

---

## Result and Analysis

### Problem 1 Result

程式產生的 Problem 1 matching 已通過 stability check：

```
P1 stable: True
```

這符合 men-proposing DA 的理論性質。只要 preference list 是 complete list，而且每個 man 持續向尚未提過的 woman proposal，演算法最後會停在 stable matching。

## Problem 2 Result

本次三組 matching 的檢查結果是：

```
Matching_1: STABLE
Blocking Pairs: []

Matching_2: UNSTABLE
Blocking Pairs: 32 pairs

Matching_3: UNSTABLE
Blocking Pairs: 23 pairs
```

我覺得這題重點不是只判斷 stable/unstable，而是必須把所有 blocking pairs 列出來。因此實作上不是找到第一個 blocking pair 就停，而是完整掃過所有 man 和他更喜歡的 women。

## Problem 3 Result

Problem 3 找到的 fairest stable matching total rank sum 是：

```
59
```

並且驗證結果為：

```
P3 stable: True
```

這題最重要的是，它不是直接暴力枚舉全部 permutation，而是先用 DA 取得 stable upper bound，再用 stability pruning 和 lower bound pruning 大幅縮小搜尋空間。

從輸出可以看到：

```
visited = 498
stability_prunes = 352
bound_prunes = 85
```

這代表大部分不可能的分支都在很早期就被剪掉，而不是等整個 matching 排完後才檢查。

## Problem 4 Result

Problem 4 的 many-to-one matching 也通過 stability check：

```
P4 stable: True
```

此外，輸出檔共有 50 筆 student assignment，加上 header 共 51 行，符合 50 位 students 都被分配的需求。每間 school 剛好保留 10 位 student，也符合 quota = 10 的設定。

---

## Verification

我最後有跑以下指令：

```
python3 -m py_compile 314581030_HW4/src/main.py
```

確認 Python 語法沒有問題。

接著執行主程式：

```
python3 -B 314581030_HW4/src/main.py
```

終端機輸出為：

```
P1 stable: True
P2 cases checked: Matching_1=0 BP, Matching_2=32 BP, Matching_3=23 BP
P3 stable: True
P3 fair rank sum: 59 (visited=498, stability_prunes=352, bound_prunes=85)
P4 stable: True
```

另外我也跑了 Problem 3 的獨立 brute-force checker：

```
python3 -B 314581030_HW4/src/hw3_checker.py --progress-every 50000000
```

最後輸出重點為：

```
solution_stable=True
solution_cost=59
brute_cost=59
checked_permutations=479001600
stable_matchings=5
elapsed_seconds=316.72
HW3/P3 checker passed: P3 solution matches brute-force optimum.
```

最後檢查輸出檔行數：

```
314581030_HW4_p1.csv: 31 lines
314581030_HW4_p2.csv: 8 lines
314581030_HW4_p3.csv: 13 lines
314581030_HW4_p4.csv: 51 lines
```

其中 P1 是 30 pairs + header，P3 是 12 pairs + header，P4 是 50 assignments + header。

---

## My Understanding

這次作業其實四題都圍繞在同一件事情：preferences 如何決定 stable outcome。

Problem 1 是最標準的 one-to-one DA，所以重點在於理解 proposal、hold、reject 這三件事。它的穩定性來自於 rejected proposer 往下一個選項走，而 receiver 永遠只保留目前看過最好的 proposer。

Problem 2 則是反過來檢查一個結果是不是 stable。這題讓我比較清楚看到 stable matching 的定義其實不是「大家都拿到很好的選項」，而是「不存在兩個人有動機私下離開原本 matching 去配對」。

Problem 3 比較像是在 stable matching 的集合裡再加上一個 fairness objective。DA 可以保證 stable，但不一定保證 total rank sum 最小，所以需要額外搜尋。不過如果完全暴力搜尋，又沒有利用 stable matching 的限制，因此我用 pruning 方式把不可能 stable 或不可能更好的分支提前排除。

Problem 4 則是把 DA 從一對一改成 many-to-one。直覺上 school 不再只 hold 一個人，而是 hold 一個大小最多為 quota 的集合。除此之外，核心精神其實仍然是 Deferred Acceptance：proposal 不是立即 final，而是暫時保留，直到後面可能被更好的 candidate 取代。